作业 _Week7

20244227004 蔡博凯

2024 年 10 月 25 日

第一题 用 python 实现拉普拉斯修正的朴素贝叶斯分类器，并以表 3 为训练集，对“测 1”样本进行判别。(请做好充分的注释，提供代码及结果)

表3

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	0.634	0.264	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	0.608	0.318	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	0.556	0.215	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	0.403	0.237	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	0.481	0.149	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	0.437	0.211	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	0.243	0.267	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	0.245	0.057	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	0.343	0.099	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	0.639	0.161	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	0.657	0.198	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	0.360	0.370	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	0.593	0.042	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	0.719	0.103	否

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
测 1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	?

解答

概述

本代码实现了一个朴素贝叶斯分类器，支持离散特征和连续特征的处理。在训练阶段，计算每个类别的先验概率和条件概率：对于离散特征，使用频率统计和拉普拉斯修正；对于连续特征，计算均值和标准差。在预测阶段，通过对每个类别计算后验概率，基于高斯分布对连续特征进行概率估计，最终选择概率最大的类别作为预测结果。

数据集与代码

可在附录查看数据集，请参考 西瓜数据集。

可在附录查看完整代码，请参考 朴素贝叶斯分类器。

运行结果

```
# 进行预测
prediction = model.predict(test_data)  # 对测试样本进行预测
print(f"预测结果: {prediction[0]}")  # 输出预测结果

[19]  ✓ 0.0s

…  预测结果: 是
```

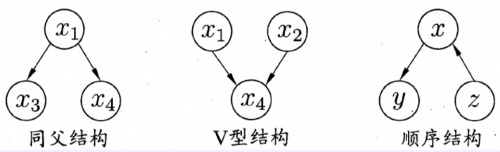
图 1: 测 1 样本预测结果

结论

可以看到，预测结果为是，与训练集第一个数据的结果相同，可以视为预测正确。

第二题

考虑下图，试证明：在同父结构中，若x_1的取值未知，则 $x_3 \perp\!\!\!\perp x_4$ 不成立；在顺序结构中，$y \perp z \mid x$, 但 $y \perp\!\!\!\perp z$ 不成立。



解答

1. 首先考虑同父结构

根据贝叶斯网结构，其联合分布可以表示为：

$$P(x_1, x_3, x_4) = P(x_1)P(x_3|x_1)P(x_4|x_1)$$

在给定 x_1 时，

$$P(x_3, x_4|x_1) = \frac{P(x_1, x_3, x_4)}{P(x_1)} = \frac{P(x_1)P(x_3|x_1)P(x_4|x_1)}{P(x_1)} = P(x_3|x_1)P(x_4|x_1)$$

即同父结构中 x_3, x_4 关于 x_1 条件独立；

在 x_1 取值未知时有：

$$P(x_3, x_4) = \sum_{x_1} P(x_1, x_3, x_4) = \sum_{x_1} P(x_1)P(x_3|x_1)P(x_4|x_1)$$

是推不出 $P(x_3, x_4) = P(x_3)P(x_4)$ 的，所以同父结构中 x_3, x_4 关于 x_1 边际独立不成立。

2. 再考虑顺序结构

根据贝叶斯网结构，其联合分布有：

$$P(x, y, z) = P(z)P(x|z)P(y|x)$$

给定 x 时，

$$P(y, z|x) = \frac{P(x, y, z)}{P(x)} = \frac{P(z)P(x|z)P(y|x)}{P(x)} = \frac{P(x, z)P(y|x)}{P(x)} = P(z|x)P(y|x)$$

即顺序结构中 y, z 关于 x 条件独立；

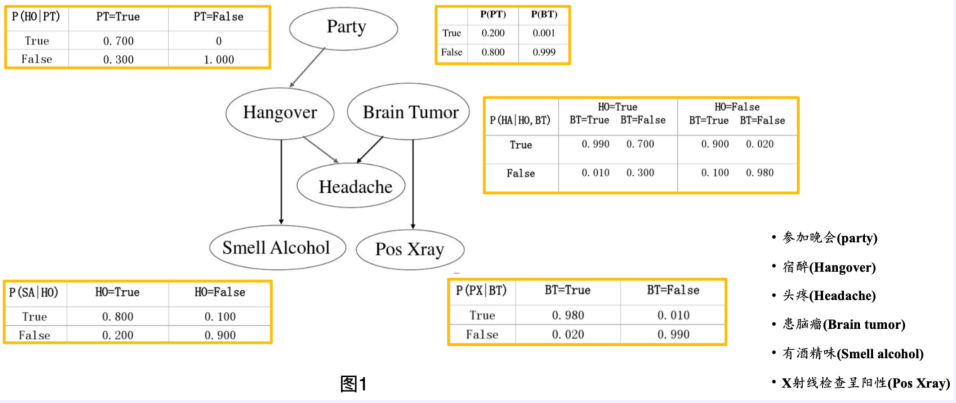
在 x 取值未知时有：

$$P(y, z) = \sum_{x} P(x, y, z) = \sum_{x} P(z)P(x|z)P(y|x)$$

同样推不出 $P(y, z) = P(y)P(z)$ ，所以顺序结构中，同样 y, z 关于 x 边际独立不成立。

第三题

试根据图1中的贝叶斯网计算：（1）发生头痛的概率$P(HA)$，（2）假设X射线成阳性，患脑瘤的概率$P(BT|PX)$。请写明计算步骤，勿直接代入数字。



P(HO|PT)	PT=True	PT=False
True	0.700	0
False	0.300	1.000

	P(PT)	P(BT)
True	0.200	0.001
False	0.800	0.999

P(HA|HO,BT)	HO=True BT=True	HO=True BT=False	HO=False BT=True	HO=False BT=False
True	0.990	0.700	0.900	0.020
False	0.010	0.300	0.100	0.980

P(SA|HO)	HO=True	HO=False
True	0.800	0.100
False	0.200	0.900

P(PX|BT)	BT=True	BT=False
True	0.980	0.010
False	0.020	0.990

图1

解答

为了描述方便，对于任何一个结点 Point:

- $P(+Point)$ 表示 Point 发生的概率
- $P(-Point)$ 表示 Point 不发生的概率

(1) 计算发生头痛的概率 P(+HA)

根据全概率公式，可得

$$\begin{aligned} P(+HA) = \quad & P(+HO) \times P(+BT) \times P(+HA|+HO,+BT) \\ & + P(+HO) \times P(-BT) \times P(+HA|+HO,-BT) \\ & + P(-HO) \times P(+BT) \times P(+HA|-HO,+BT) \\ & + P(-HO) \times P(-BT) \times P(+HA|-HO,-BT) \end{aligned} \tag{1}$$

由于 HO 依赖于 PT, 还需要依据全概率公式求解

$$P(+HO) = P(+PT) \times P(+HO|+PT) + P(-PT) \times P(+HO|-PT) \tag{2}$$

$$P(-HO) = 1 - P(+HO) \tag{3}$$

代入表中数据可求解式子 (2) 和 (3)

$$P(+HO) = 0.2 \times 0.7 + 0.8 \times 0 = 0.14$$

$$P(-HO) = 1 - 0.14 = 0.86$$

因此式子 1 中所有数据已获知，代入数据可求解

$$\begin{aligned} P(+HA) &= 0.14 \times 0.001 \times 0.99 + 0.14 \times 0.999 \times 0.7 \\ &\quad + 0.86 \times 0.001 \times 0.9 + 0.86 \times 0.999 \times 0.02 \\ &\approx 0.116 \end{aligned}$$

此外，还可以算出 HA 不发生的概率，根据全概率公式，可得

$$P(-HA) = 1 - 0.116 = 0.884$$

因此，在没有任何诱因的情况下，头痛发生的概率是 0.116，不头痛的概率是 0.884

(2) 求解 X 射线呈阳性的前提下患脑癌的概率 P(+BT|+PX)

根据条件概率公式，可得

$$P(+BT|+PX) = \frac{P(+PX|+BT) \times P(+BT)}{P(+PX)} \tag{4}$$

根据全概率公式，可得

$$P(+PX) = P(+PX|+BT) \times P(+BT) + P(+PX|-BT) \times P(-BT) \tag{5}$$

代入表中数据可求解式子 (5)

$$P(+PX) = 0.98 \times 0.001 + 0.01 \times 0.999 \approx 0.011$$

因此式子 4 中所有数据已获知，代入数据可求解

$$P(+BT|+PX) = \frac{0.98 \times 0.001}{0.011} \approx 0.089$$

因此，X 射线呈阳性的前提下患脑癌的概率为 0.089(概率较低)

参考文献

[1] https://github.com/hanmq/MachineLearning_Zhouzhihua_ProblemSets

[2] https://wenku.baidu.com/view/bab21ac77f1cfad6195f312b3169a4517723e5dd.html?_wkts_=1729772511456&needWelcomeRecommand=1

A 西瓜数据集

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.46	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	0.634	0.264	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	0.608	0.318	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	0.556	0.215	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	0.403	0.237	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	0.481	0.149	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	0.437	0.211	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	0.243	0.267	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	0.245	0.057	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	0.343	0.099	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	0.639	0.161	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	0.657	0.198	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	0.36	0.37	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	0.593	0.042	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	0.719	0.103	否

B 朴素贝叶斯分类器

```python
import pandas as pd
import numpy as np

class NaiveBayes:
    def __init__(self):
        self.class_prior = {}  # 存储每个类别的先验概率
        self.likelihood = {}    # 存储每个类别的条件概率
        self.labels = []        # 存储所有类别标签

    def fit(self, X, y):
        # 获取所有类别
        self.labels = y.unique()
        for label in self.labels:
            # 计算先验概率
            # 类别 label 的先验概率
            self.class_prior[label] = (y == label).mean()
            self.likelihood[label] = {}  # 初始化条件概率字典
            label_data = X[y == label]  # 获取该类别的特征数据
            for column in X.columns:
                if X[column].dtype == 'object':  # 离散特征
                    # 统计每个特征值出现的次数
                    value_counts = label_data[column].
                        value_counts()
                    # 该类别的总样本数
                    total_count = len(label_data)
                    # 使用拉普拉斯修正计算条件概率
                    self.likelihood[label][column] = {
                        value: (count + 1) / (total_count + len(
                            value_counts))
                        for value, count in value_counts.items()
                    }
```

```
30                 else:  # 连续特征
31                     # 计算均值和标准差
32                     mean = label_data[column].mean()
33                     std = label_data[column].std()
34                     # 存储均值和标准差
35                     self.likelihood[label][column] = (mean, std)
36
37     def predict(self, X):
38         predictions = []  # 用于存储每个测试样本的预测结果
39         for _, row in X.iterrows():  # 遍历每一行测试数据
40             posteriors = {}  # 存储后验概率
41             for label in self.labels:  # 对每个类别计算后验概率
42                 # 初始化后验概率为先验概率的对数
43                 posterior = np.log(self.class_prior[label])
44                 for column in X.columns:
45                     if X[column].dtype == 'object':  # 离散特征
46                         value = row[column]  # 当前特征值
47                         # 获取条件概率，若特征值未见则用平滑值
48                         likelihood_value = self.likelihood[label
                               ].get(column, {}).get(value, 1 / (len
                               (y_train) + 1))
49                         # 累加对数条件概率
50                         posterior += np.log(likelihood_value)
51                     else:  # 连续特征
52                         mean, std = self.likelihood[label][
                               column]
53                         # 使用高斯分布计算概率密度
54                         likelihood_value = (1 / (std * np.sqrt(2
                                * np.pi))) * np.exp(-0.5 * ((row[
                               column] - mean) / std) ** 2)
55                         # 累加对数条件概率
56                         posterior += np.log(likelihood_value)
57                 # 保存该类别的后验概率
```

```
58                  posteriors[label] = posterior
59              # 找到具有最大后验概率的类别
60              predictions.append(max(posteriors, key=posteriors.
                    get))
61          return predictions  # 返回所有预测结果
62
63  if __name__ == '__main__':
64      # 从 CSV 文件读取数据
65      df = pd.read_csv("watermelon3_0_Ch.csv")
66      # 分离特征和标签
67      # 特征集，去掉“编号”和“好瓜”列
68      X_train = df.drop(columns=["编号", "好瓜"])
69      y_train = df["好瓜"]  # 标签集
70      # 实例化并训练模型
71      model = NaiveBayes()  # 创建朴素贝叶斯模型实例
72      model.fit(X_train, y_train)  # 用训练数据训练模型
73      # 测试数据
74      test_data = pd.DataFrame({
75          "色泽": ["青绿"],  # 测试样本的特征
76          "根蒂": ["蜷缩"],
77          "敲声": ["浊响"],
78          "纹理": ["清晰"],
79          "脐部": ["凹陷"],
80          "触感": ["硬滑"],
81          "密度": [0.697],
82          "含糖率": [0.46]
83      })
84      # 进行预测
85      prediction = model.predict(test_data)  # 对测试样本进行预测
86      print(f"预测结果: {prediction[0]}")  # 输出预测结果
```